

ragscore

Mesurer un systeme RAG au lieu de l'affirmer bon.

Documentation complete : les deux facons de mesurer, le format du jeu de questions, chaque indicateur explique et calcule, les couts, la reference de configuration.

Le principe

ragscore est un arbitre. Il ne connait pas le jeu, il connait les regles. Vous lui donnez une liste de questions avec les bonnes reponses, et un systeme capable de repondre. Il note **la recuperation et la generation separement**, de facon **deterministe**, et produit un rapport **renotable sans rappeler le systeme**.

Il n'appelle lui-meme aucun modele et ne connait ni votre base, ni votre langue, ni votre domaine.

Pourquoi separer les deux etages

Un score unique ne dit pas ou corriger. Si le bon document n'est pas dans le premier etage de recuperation, aucun reclassement ne l'y fera apparaitre, et travailler le reranker est une perte de temps. A l'inverse, un systeme qui trouve le bon document et le resume mal n'a pas le meme probleme qu'un systeme qui ne le trouve pas. La distinction dit lequel des deux vous avez.

Pourquoi une notation deterministe

Aucune valeur attendue n'est appreciee par un modele : elle est presente au caractere pres, a la mise en forme pres, ou elle ne l'est pas. Un juge LLM se justifie pour une reponse ouverte, jamais pour un chiffre. Une notation qui varie d'une execution a l'autre ne permet de comparer ni deux versions, ni deux modeles.

Les trois objets

Objet	Ce qu'il porte	Qui l'ecrit
Case	une question, les documents qui y repondent, ce que la reponse doit contenir	vous
Answer	le texte produit et les documents retenus par chaque etage	votre systeme
CaseOutcome	la note : rangs, presence des valeurs, refus	ragscore

Le contrat a remplir tient en une methode : `answer(question)` qui rend une `Answer`. Ni interface a heriter, ni configuration.

Installation

```
cd ~/Desktop/ragscore
uv sync --extra ui --extra engine
```

Extra	Ce qu'il ajoute	Necessaire pour
(aucun)	httpx seulement	utiliser ragscore comme bibliotheque

Extra	Ce qu'il ajoute	Necessaire pour
ui	starlette, uvicorn	l'interface locale
engine	psycpg, pgvector, voyageai, anthropic	le moteur integre
docs	reportlab	regenerer ce PDF

Toutes les commandes se prefixent par `uv run`. Le programme est installe dans `.venv/bin`, qui n'est pas dans le PATH du terminal : sans `uv run`, le shell repond `command not found`. L'alternative est source `.venv/bin/activate`, une fois par session.

Les deux facons de mesurer

Elles ne repondent pas a la meme question, et il faut les deux.

Sorte	Vous branchez	La question a laquelle elle repond	Prerequis
vector moteur integre	une base vectorisee et deux cles	cette approche est-elle bonne ? Ce decoupage, ce reranker, ce modele d'embeddings	la base
http API a brancher	l'URL de votre application	mon produit est-il bon ? C'est le chiffre que l'on publie	la base et l'application

Brancher l'API reste la mesure qui compte avant de vendre, et aucune reimplementation ne la remplace : deux chemins de requete divergent des qu'on touche a l'un, et l'evaluation finit par noter autre chose que ce qu'on livre. Le moteur integre repond a l'autre question, celle que l'on se pose en amont, quand l'application n'existe pas encore ou que l'on compare deux strategies sans rien deployer.

Les deux chiffres ne sont pas comparables entre eux. Le moteur integre utilise votre corpus, mais pas votre chemin de requete : ni vos expansions, ni votre consigne systeme, ni vos verifications. Un rapport issu du moteur ne doit jamais etre presente comme la mesure du produit.

Le moteur integre (kind: vector)

ragScore execute la chaine complete : il vectorise la question, cherche dans la base, reclasse, puis genere. Deux etages sont releves, nommes recherche et rerank.

```
{
  "kind": "vector",
  "name": "mon corpus",
  "databaseUrl": "{{env:DATABASE_URL}}",
  "fromClause": "chunk JOIN document ON document.id = chunk.documentid",
  "identifierColumn": "document.code",
  "textColumn": "chunk.content",
  "embeddingColumn": "chunk.embedding",
  "searchLimit": 40,
  "rerankKeep": 8,
  "maxPassages": 14,
  "embeddingModel": "voyage-3.5",
  "rerankModel": "rerank-2.5",
  "generationModel": "claude-sonnet-5"
}
```

Quand l'identifiant vit dans la meme table que le texte, table suffit et fromClause reste vide. Sinon on donne la relation avec sa jointure, et les colonnes se qualifient. fromClause et whereClause sont du SQL assume : de la configuration, jamais une valeur saisie par un utilisateur final. Les noms de table et de colonnes, eux, passent par des identifiants SQL composes, jamais concatenes.

La consigne donnee au modele decide du score. Celle par defaut demande de citer l'identifiant et de refuser hors corpus. Sans cette consigne, tous les cas de refus echouent et la citation n'est jamais mesurable. La modifier est legitime : c'est souvent ce que l'on cherche a mesurer.

L'API a brancher (kind: http)

Deux formes de reponse sont couvertes. En **flux SSE**, chaque evenement est associe a son role. En **JSON**, on lit le texte et les documents par chemin pointe.

```
{
  "kind": "http",
  "name": "mon assistant (local)",
  "url": "http://127.0.0.1:3000/api/chat",
  "mode": "sse",
  "body": {"question": "{{question}}", "debug": true},
  "textSource": "text",
  "textField": "delta",
  "stages": [
    {"name": "candidats", "source": "candidates", "itemField": null},
    {"name": "rerank", "source": "sources", "itemField": "code"}
  ],
  "usage": {"source": "done"},
  "authentication": {
    "url": "http://127.0.0.1:3000/api/auth/connexion",
    "body": {"email": "{{env:EVALUATION_EMAIL}}",
    "password": "{{env:EVALUATION_PASSWORD}}"}
  }
}
```

`itemField` sert quand l'API renvoie des objets plutot que des chaines : [{"code": "A"}] se lit avec "code", ["A"] sans rien. L'etape authentication est rejouee une fois, et ses cookies sont conservees pour tous les appels suivants.

En mode JSON, textSource devient un chemin (data.answer) et chaque etage lit une liste par chemin.

Le jeu de questions

Un fichier JSON Lines, une ligne par cas. Le depot dans l'interface accepte aussi le CSV d'un tableur et un JSON contenant une liste.

```
{
  "id": "montant-granules",
  "question": "Combien de kwh cumac pour une chaudiere a granules en zone H1 ?",
  "relevant_ids": ["BAR-TH-113"],
  "must_include": ["41 300 kwh cumac"],
  "must_include_variants": {"41 300 kwh cumac": ["41300 kwh cumac"]},
  "must_not_include": [],
  "category": "montant"
}
```

Champ	Role
id	identifiant unique. Deux cas de meme id s'ecraserait a la renotation, le chargement echoue donc
question	ce qui est envoye au systeme, tel quel
relevant_ids	les documents qui repondent. Vide = cas de refus
must_include	ce que la reponse doit contenir, typiquement une valeur chiffrée
must_include_variants	orthographes equivalentes acceptees, declarees une par une
must_not_include	les pieges, surtout sur un cas de refus
category	etiquette libre servant a ventiler le score (usage, secteur, difficulté)

Le cas de refus

Un cas dont `relevant_ids` est vide est un cas de refus : la question sort du corpus, et la seule bonne reponse est de dire qu'on ne sait pas. C'est la **structure** du cas qui porte l'information, pas une categorie nommée dans une langue particuliere.

Il est reussi a deux conditions : le systeme dit qu'il ne sait pas, ET il ne laisse echapper aucune valeur interdite. Un systeme qui refuse poliment tout en citant un montant plausible echoue, et c'est le comportement qui coute un client.

Les variantes d'orthographe

Une variante non declaree n'est pas acceptee. Le jeu reste explicite, jamais devinatoire : si 3.7 kwh doit valoir 3,7 kwh, cela s'ecrit. La comparaison tolere en revanche toujours la mise en forme : gras, echappements Markdown, espaces insecables, accents et casse.

La verite terrain se verifie contre le corpus, jamais de memoire. ragscore ne peut pas le faire a votre place, il ne connait pas votre corpus. Mais c'est la condition pour que le score veuille dire quelque chose : un jeu de test faux donne confiance a tort, ce qui est pire que pas de jeu du tout. Sur le corpus CEE, chaque fragment attendu est verifie present dans la fiche citee avant d'entrer dans le jeu, et la generation echoue bruyamment sinon.

Les indicateurs, un par un

Les cas de refus sont **exclus** de tous les rappels et de tous les rangs : ils n'ont aucun document pertinent, donc les inclure ferait monter le score sans rien mesurer. Ils sont notes a part, par « refus corrects ».

Rappel par etage

Pour chaque etage de recuperation : la part des cas ou **au moins un** document pertinent s'y trouve. Un etage est une etape nommee de votre chaine, dans l'ordre ou le systeme l'applique.

Comment le lire. C'est un entonnoir. Si le rappel chute entre deux etages, c'est le second qui perd le document. S'il est deja bas au premier, le probleme est dans la recherche ou dans l'indexation, et aucun reclassement ne le rattrapera.

Ce que vous observez	Ce que cela signifie	Ou chercher
recherche 100 %, rerank 100 %	la recuperation n'est pas votre probleme	la generation, ou le jeu de test
recherche 100 %, rerank 78 %	le reranker jette de bons documents	le modele de reclassement, le seuil, le nombre garde
recherche 62 %	le document n'est jamais candidat	le decoupage, les embeddings, le nombre cherche

MRR, rang moyen inverse

Le rappel dit si le document est la. Le MRR dit **a quelle place**. Pour chaque cas on prend l'inverse du rang du meilleur document pertinent dans l'etage final, puis on fait la moyenne. Un document absent compte zero.

$$MRR = \text{moyenne de } (1 / \text{rang})$$

Rang du bon document	Contribution
1 ^{er}	1,000
2 ^e	0,500
3 ^e	0,333
5 ^e	0,200
10 ^e	0,100
absent	0,000

Exemple reel. Sur les 45 cas du jeu CEE qui attendent une fiche, 42 la placent au premier rang et 3 au second :

$$\begin{aligned} MRR &= (42 \times 1,000 + 3 \times 0,500) / 45 \\ &= 43.5 / 45 \\ &= 0.967 \end{aligned}$$

Comment le lire. 1,000 signifie « toujours en tete ». La chute est brutale : passer du premier au second rang coute la moitie. C'est voulu, parce qu'un document cite en deuxieme position pese deja beaucoup moins dans ce que le modele lit. En dessous de 0,7, la recuperation ramene le bon document mais mal

classe, et le modele travaille dans le bruit.

nDCG, qualite du classement

Meme idee que le MRR, avec une decroissance logarithmique au lieu d'inverse, et surtout la capacite de compter **plusieurs** documents pertinents. Le resultat est normalise par le classement parfait, donc toujours entre 0 et 1.

gain = somme de ($1 / \log_2(\text{rang} + 1)$), puis divise par le classement parfait

Rang	MRR y voit	nDCG y voit
1 ^{er}	1,000	1,000
2 ^e	0,500	0,631
3 ^e	0,333	0,500
5 ^e	0,200	0,387

Sur le meme jeu CEE : $nDCG = (42 \times 1,000 + 3 \times 0,631) / 45 = 0.975$, contre 0,967 pour le MRR. L'ecart est normal : le nDCG punit moins durement un document trouve en deuxieme position.

Quand il compte vraiment. Des qu'un cas a plusieurs documents pertinents. Le MRR ne regarde alors que le meilleur et ignore si vous avez trouve les autres ; le nDCG les compte tous. Sur un jeu a un seul document par question, les deux disent presque la meme chose.

Document cite

La part des reponses qui mentionnent litteralement l'identifiant attendu. La recherche se fait dans le **texte brut**, sans normalisation : un identifiant est sensible a la casse et aux tirets, et bar - th - 113 n'est pas BAR - TH - 113.

Comment le lire. Un ecart entre « trouve apres rerank » et « document cite » signifie que le systeme a le bon extrait sous les yeux et ne le nomme pas. C'est un probleme de consigne, pas de recuperation. Pour un professionnel qui doit verifier une source, c'est redhibitoire.

Valeur attendue

La part des reponses contenant tous les fragments de must_include. C'est l'indicateur le plus severe et le plus utile : il ne juge pas le style, il verifie le chiffre.

La comparaison normalise la mise en forme (gras, echappements Markdown, espaces insecables, accents, casse) et rien d'autre. 19 900 ne passe pas pour 19 800.

Refus corrects

Calcule sur les seuls cas de refus. Reussi si le systeme dit qu'il ne sait pas **et** ne produit aucune valeur interdite. La detection du refus se fait par motifs, et c'est la partie fragile de la notation : un refus parfaitement formule mais non reconnu compte comme un echec et sous-estime le systeme. Les motifs par default tolerent les incises, comme « je ne peux *donc* pas repondre ».

Les motifs sont propres a une langue et s'injectent : un jeu francais et un jeu anglais n'utilisent pas les memes.

Aucune valeur interdite

Calcule sur **tous** les cas, refus compris. C'est le garde-fou contre l'invention : une valeur que la reponse ne devait jamais contenir et qui s'y trouve.

Cas reussis

Le score global, celui qu'on annonce. Un cas ordinaire est reussi si le document est trouve, que toutes les valeurs attendues sont presentes et qu'aucune valeur interdite ne l'est. Un cas de refus est reussi selon la regle ci-dessus.

Ventilation par categorie

Les memes chiffres, decoupees par l'etiquette category. C'est souvent la seule vue qui montre quelque chose : un score global de 94 % peut cacher un type de question a 60 %. Sur le corpus CEE, les categories separent routage, montant, duree, condition, version et refus.

Ce qu'aucun de ces indicateurs ne mesure

La qualite d'une reponse ouverte. « Explique-moi la condition de mise en oeuvre » n'a pas de fragment attendu verifiable, donc seule la recuperation y est notee. C'est la limite assumee de la notation deterministe, et la raison pour laquelle un juge LLM viendra la completer, valide au préalable contre un annotateur humain.

Ils ne mesurent pas non plus la latence, la robustesse aux questions malveillantes, ni le comportement sur un corpus qui change.

Ce que ca consomme

ragScore n'appelle aucun modèle. Ce qui coûte, c'est le système mesure : une question déclenche une vectorisation, un reclassement et une génération, donc **deux fournisseurs**.

Action	Appels au système	Coût
Lancer la mesure, 50 questions	50	0,80 à 1,20 \$
Tester sur une question	1	environ 0,02 \$
Renoter un rapport	0	gratuit
Afficher un rapport	0	gratuit
Toute la navigation dans l'interface	0	gratuit

Tarifs de génération, en dollars par million de jetons

Modèle	Entrée	Sortie	Mesure sur le jeu CEE	Coût des 50
claude-sonnet-5	2 \$	10 \$	50 / 50	environ 0,80 \$
claude-haiku-4-5	1 \$	5 \$	49 / 50	environ 0,30 \$
claude-opus-5	5 \$	25 \$	non mesure	environ 2 \$

L'unique échec de Haiku 4.5 portait sur un raisonnement dans un tableau déjà récupéré, pas sur une récupération. Il convient très bien pour éprouver un câblage ou itérer sur un découpage ; le chiffre que l'on publie se mesure avec le modèle que l'on livre.

Le coût affiche sous-estime la réalité. Il ne compte que les jetons de génération. Les appels de vectorisation et de reclassement ne sont pas comptés, alors que le total se présente comme complet. L'écart est faible, la génération pesant beaucoup plus lourd, mais il n'est pas nul.

La renotation, et pourquoi elle existe

Le rapport sépare **ce que le système a répondu** de **la note qu'on lui a mise**. Quand la correction porte sur le harnais (détection du refus, normalisation, variante acceptée, fragment attendu mal recopié) et non sur le système, les réponses sont identiques au caractère près : les renoter ne coûte rien.

Ce n'est pas un confort. Trois exécutions complètes d'un jeu de 50 questions ont été gaspillées faute de ce mode.

Vous avez changé...	Renoter suffit ?
une valeur attendue, une variante, un motif de refus	oui, et c'est gratuit
l'étiquette ou l'intitulé d'un cas	oui
le découpage, les embeddings, le reranker, le prompt, le modèle	non, il faut remesurer
ajoute de nouveaux cas au jeu	non pour ceux-là, ils seront ignorés

Les secrets

Une declaration de systeme ne stocke **jamais** une valeur secrete. On y ecrit `{{env:NOM}}`, et la valeur est lue dans l'environnement du processus au moment de l'appel. Une variable manquante est signalee par son nom, pas par une erreur obscure.

```
DATABASE_URL=... VOYAGE_API_KEY=... ANTHROPIC_API_KEY=... uv run ragscore ui

# ou, si elles vivent deja dans un fichier :
set -a && source ~/mon-projet/.env && set +a && uv run ragscore ui
```

Garde-fou	Ce qu'il fait
<code>{{env:NOM}}</code>	le fichier porte le nom de la variable, jamais sa valeur
droits 600	les declarations enregistrees sont lisibles par leur seul proprietaire
.gitignore	le dossier des declarations n'est pas versionne
boucle locale	l'interface n'ecoute que sur 127.0.0.1, jamais sur le reseau

Un mot de passe ecrit en clair dans une declaration finirait dans un fichier, puis dans une sauvegarde, puis dans un depot.

Les commandes

Commande	Ce qu'elle fait	Cout
<code>uv run ragscore ui</code>	ouvre l'interface locale sur 127.0.0.1:7654	gratuit
<code>uv run ragscore show --report R</code>	affiche le resume d'un rapport	gratuit
<code>uv run ragscore rescore --cases J --report R</code>	renote les reponses stockees. Ecrase R, sauf si <code>--out</code> est donne	gratuit
<code>uv run python examples/minimal.py</code>	exemple complet sans base ni cle d'API	gratuit

Utilisation en bibliotheque

L'interface n'est qu'une facade : tout ce qu'elle fait est faisable en Python, et rien n'est stocke dans un format qu'elle serait seule a relire.

```
from pathlib import Path
from ragscore import loadCases, runEvaluation, writeReport
from ragscore.display import formatSummary

class MonSysteme:
    def answer(self, question: str) -> Answer:
        ... # interroge votre RAG, rend le texte et les etages

cases = loadCases(Path("data/testset.jsonl"))
outcomes, summary, observations = runEvaluation(MonSysteme(), cases)
writeReport(Path("data/rapport.json"), outcomes, summary, observations)
print(formatSummary(summary))
```

Reference de configuration

Moteur integre (kind: vector)

Champ	Defaut	Role
databaseUrl	{{env:DATABASE_URL}}	connexion PostgreSQL avec pgvector
table	chunk	table portant les vecteurs, si aucune jointure
fromClause	(vide)	relation complete avec jointure, SQL assume
identifierColumn	code	colonne de l'identifiant, qualifiable
textColumn	content	colonne du texte envoye au modele
embeddingColumn	embedding	colonne du vecteur
whereClause	(vide)	filtre additionnel, SQL assume
searchLimit	40	candidats ramenés par la recherche vectorielle
rerankKeep	8	candidats gardés après reclassement
maxPassages	14	passages envoyés au modele
embeddingModel	voyage-3.5	modele de vectorisation
rerankModel	rerank-2.5	modele de reclassement
generationModel	claude-sonnet-5	modele de generation
systemPrompt	citation et refus	consigne, elle decide du score
inputPricePerMillion	2.0	tarif d'entree, pour annoncer le cout
outputPricePerMillion	10.0	tarif de sortie

API a brancher (kind: http)

Champ	Defaut	Role
url	(requis)	point d'entree interroge
mode	sse	« sse » pour un flux, « json » pour une reponse unique
method	POST	verbe HTTP
body	{"question": "{{question}}"} 	corps envoye, la question y est substituee
headers	{}	en-tetes, ou {{env:NOM}} pour un jeton

Champ	Default	Role
textSource	text	evenement (sse) ou chemin (json) du texte
textField	delta	champ du fragment, en sse uniquement
stages	[]	un etage par etape : nom, source, itemField
usage	{}	ou lire la consommation rapportee
authentication	null	appel prealable dont les cookies sont conserves
timeoutSeconds	180	delai maximal par question
errorSource	error	evenement signalant une erreur serveur

Depannage

Message	Cause et correction
command not found: ragscore	l'environnement n'est pas actif. Prefixez par uv run, ou activez avec source .venv/bin/activate
la variable d'environnement X n'est pas definie	le processus a ete lance sans elle. Relancez l'interface avec la variable
connexion a la base impossible	la base n'est pas demarree, ou databaseUrl est faux
la recherche a echoue	un nom de table ou de colonne est faux, ou la jointure manque
le systeme a repondu 500	l'application mesuree ne tourne pas, ou refuse la requete
la connexion a echoue (401)	les identifiants du connecteur sont faux ou absents
aucun texte trouve au chemin X	mode json : le chemin du texte est faux. Les cles disponibles sont listees
un etage remonte vide au test	le nom d'evenement, de chemin ou de champ est faux
aucune colonne de question reconnue	le fichier depose n'a pas de colonne question, prompt, demande ou requete
identifiants en double	deux cas portent le meme id ; ils s'ecraseront a la renotation
ne contient pas d'observations	le rapport vient d'un format anterieur et ne peut pas etre renote
le moteur integre a besoin de dependances	relancez uv sync --extra engine

Limites connues

Limite	Consequence
Pas de juge pour les reponses ouvertes	une question sans fragment verifiable n'est notee que sur la recuperation
Le cout affiche ignore la vectorisation et le reclassement	le total annonce est legerement inferieur au reel
Les ablations ne sont pas commutables	comparer avec et sans reclassement demande deux declarations et deux mesures
Le moteur integre n'ingere rien	il faut une base deja vectorisee ; ragscore ne construit pas l'index
La detection du refus est par motifs	une formulation inattendue compte comme un echec et sous-estime le systeme

La suite. Un juge LLM pour les reponses ouvertes, **valide contre un annotateur humain** avant de publier la moindre metrique de qualite : l'accord entre le juge et l'humain se mesure par le kappa de Cohen, et sans ce prealable un score de qualite ne prouve rien. Puis la commutation des etages, pour mesurer les ablations.

ragscore, 58 tests. Documentation regenee par docs/construire_documentation.py. Mise a jour du 18 septembre 2026.